

lamet-agent

LaMET/LQCD 分析智能体框架使用手册

张鑫

2026 年 7 月 20 日

摘要

lamet-agent 是一个基于 Python 的大动量有效理论 (LaMET) 格点 QCD 分析智能体框架。它通过 Manifest (清单文件) 定义全局数据源和分阶段作业列表, 利用 LLM 驱动的智能体自动执行从关联函数分析到最终物理分布函数的完整五阶段分析 workflow, 并以 NetCDF 格式输出中间数据制品。本手册详细介绍 lamet-agent 的架构设计、安装配置、Manifest 系统、五阶段 workflow、命令行接口、数据格式约定、后端配置以及核心 API 接口。

目录

1	项目概述	2
1.1	主要特性	2
1.2	五阶段 workflow	2
2	项目结构	3
3	安装	4
3.1	环境要求	4
3.2	安装步骤	4
3.3	可选依赖安装	4
4	命令行接口	5
4.1	CLI 概览	5
4.2	validate 命令	5
4.3	plan 命令	6
4.4	run 命令	6
4.5	后端模式	7

目录	2
5 Manifest 系统	7
5.1 Manifest 概述	7
5.2 metadata 字段	8
5.3 inputs 字段	8
5.4 标准 HDF5 关联函数格式	10
6 五阶段分析工作流	10
6.1 Stage 1: correlator_analysis (关联函数分析)	10
6.2 Stage 2: renormalization (重整化)	11
6.2.1 Ratio 方案	11
6.2.2 Hybrid-Ratio 方案	11
6.2.3 混合自重整化方案	11
6.3 Stage 3: fourier_transform (傅里叶变换)	12
6.4 Stage 4: perturbative_matching (微扰匹配)	12
6.5 Stage 5: extrapolation (外推)	13
6.6 阶段结束后处理	13
7 NetCDF 中间数据格式	13
7.1 EnsembleData 格式	13
7.2 典型制品链	13
7.3 Python 读写	13
7.4 外部检查	14
8 示例演示	14
8.1 示例 1: 介子 PDF CG 规范完整工作流	14
8.2 示例 2: 介子 DA GI 规范完整工作流	15
8.3 示例 3: 混合自重整化	15
8.4 示例 4: 交互式规划	16
9 代码架构详解	16
9.1 agent.py —run_agent()	16
9.2 manifest.py —模式定义	17
9.3 core/llm.py —LLM 会话后端	17
9.4 core/tools.py —工具系统	17
9.5 core/data.py —数据容器	18
10 API 接口清单	18
10.1 CLI 接口	18
10.2 Agent 核心 API	19

目录	3
10.3 Manifest API	19
10.4 数据 API	21
10.5 LLM API	21
10.6 工具系统 API	22
10.7 阶段参数合约	23
10.8 各阶段工具注册表	23
11 开发与贡献	24
11.1 运行测试	24
11.2 添加新阶段	25
11.3 添加新 LLM 提供商	25

1 项目概述

lamet-agent 是一个“Python-first”的 LaMET/LQCD 分析智能体框架。其核心思想是通过一个 Manifest 文件定义全局数据源池和分阶段作业列表，作业 ID 形成有向无环图 (DAG)：关联函数作业聚合原始数据集，后续作业通过角色命名的输入（如 `target`、`denominator`）消费上游作业 ID。

1.1 主要特性

- **五阶段分析工作流**：关联函数分析 → 重整化 → 傅里叶变换 → 微扰匹配 → 外推
- **LLM 驱动**：支持多种 LLM 后端（OpenAI API、DeepSeek、Codex、Mock），智能体自主执行各阶段分析
- **NetCDF 中间数据**：阶段间通过自描述的 NetCDF 文件传递数据，支持 bootstrap/-jackknife 重采样
- **Manifest 系统**：声明式 JSON/JSONC 清单文件定义完整的分析流水线
- **参数合约**：严格的阶段参数校验，拒绝未知键和类型错误
- **混合自重整化**：支持 ratio 方案、hybrid-ratio 方案和混合自重整化方案
- **规划模式**：lamet-agent plan 命令通过 LLM 交互式地生成和修复 Manifest

1.2 五阶段工作流

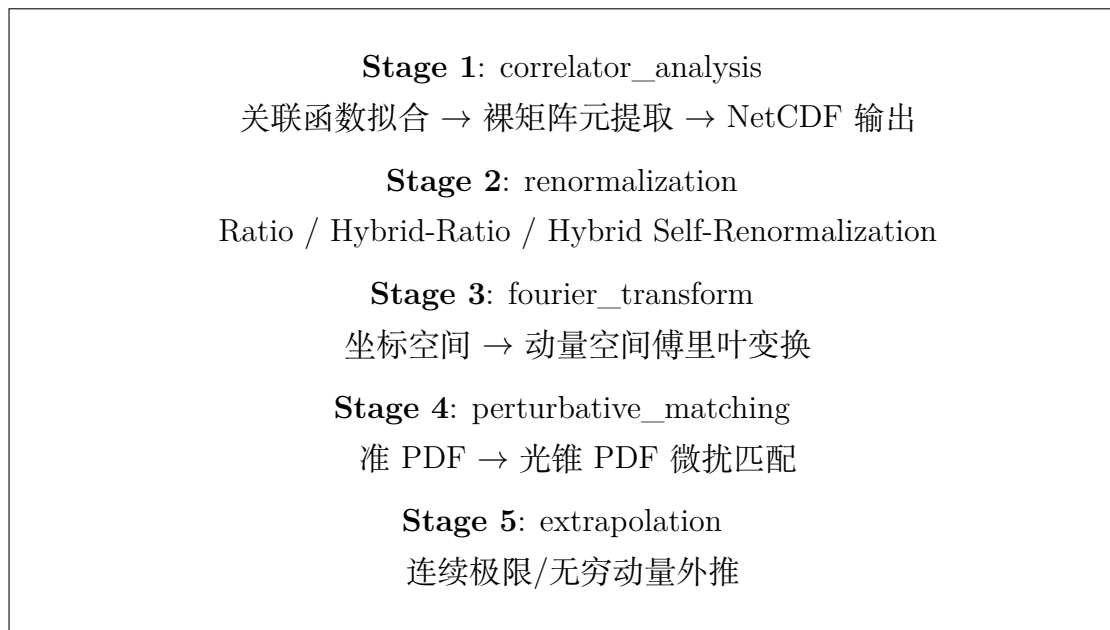


图 1: lamet-agent 五阶段分析工作流

2 项目结构

```

1  .
2  pyproject.toml          # 项目元数据和依赖声明
3  README.md              # 完整项目文档
4  examples/              # 示例和参考数据
5      fake_data/          # 模拟关联函数数据生成器
6      sample_manifest.jsonc # 带注释的参考Manifest
7      pion_pdf_cg_manifest.json # 介子PDF CG规范工作流
8      pion_pdf_gi_manifest.json # 介子PDF GI规范工作流
9      pion_da_gi_manifest.json # 介子DA GI规范工作流
10     kaon_da_gi_manifest.json # K介子DA GI规范工作流
11  src/lamet_agent/       # 源代码
12     __init__.py         # 包初始化
13     agent.py             # run_agent() — 主循环
14     cli.py               # CLI入口 (validate/run/plan)
15     kernels.py           # 内置匹配核函数
16     manifest.py          # Manifest模式定义和验证
17     manifest_params.py   # 参数合约注册表
18     core/                # 核心模块
19         llm.py           # 可插拔LLM会话后端
20         prompting.py      # 提示词构建
21         tools.py          # 工具注册和参数准备
22         trace.py          # ReAct风格追踪
23         data.py           # 类型化数据容器 (EnsembleInfo
24     /EnsembleData)
25         stages.py         # 阶段ID到包的映射
26         reporting.py      # 阶段报告生成
27         resampling.py     # 重采样 (jackknife/bootstrap)
28     )
29     plotting.py          # 出版级绑图
30     banner.py            # 启动横幅
31     planning/            # 交互式规划模块
32     stages/              # 阶段实现
33         correlator/       # Stage 1: 关联函数分析
34         renorm/           # Stage 2: 重整化
35         fourier/          # Stage 3: 傅里叶变换
36         matching/        # Stage 4: 微扰匹配
37         extrapolation/    # Stage 5: 外推
38         review/          # 跨阶段一致性审查

```

37

tests/unit/

单元测试

Listing 1: lamet-agent 项目目录结构

3 安装

3.1 环境要求

表 1: 运行环境要求

组件	要求
Python	≥ 3.10
核心依赖	numpy (≥ 2.0), pydantic (≥ 2.8), typer (≥ 0.12), tqdm (≥ 4.68)
分析依赖 (analysis)	gvar (≥ 12.1), lsqfit (≥ 13.0), scipy (≥ 1.13), h5py (≥ 3.11), matplotlib (≥ 3.8), xarray (≥ 2024.1), netCDF4 (≥ 1.7)
开发依赖 (dev)	pytest (≥ 8.0)
Codex 依赖 (codex)	openai-codex ($\geq 0.1.0b3$)

3.2 安装步骤

```

1 # 1. 克隆仓库
2 git clone https://github.com/Greyyy-HJC/lamet-agent.git
3 cd lamet-agent
4
5 # 2. 创建虚拟环境
6 python3 -m venv .venv
7 source .venv/bin/activate
8
9 # 3. 安装 (含分析和开发依赖)
10 pip install -e ".[dev,analysis]"
11
12 # 4. 验证安装
13 lamet-agent --help

```

Listing 2: lamet-agent 安装流程

3.3 可选依赖安装

```

1 # 仅核心安装
2 pip install -e .
3
4 # 含分析依赖 (NetCDF, gvar, lsqfit, matplotlib)
5 pip install -e ".[analysis]"
6
7 # 含Codex后端支持
8 pip install -e ".[codex]"
9
10 # 完整安装
11 pip install -e ".[dev,analysis,codex]"

```

Listing 3: 按需安装可选依赖

4 命令行接口

4.1 CLI 概览

lamet-agent 通过 lamet-agent 命令提供三个子命令：

```

1 # 验证Manifest文件
2 lamet-agent validate <manifest.json>
3
4 # 交互式规划 (通过LLM修复和完善Manifest)
5 lamet-agent plan <manifest.json> --backend api --model deepseek/
    deepseek-chat
6
7 # 运行分析 workflow
8 lamet-agent run <manifest.json> --backend api --model deepseek/
    deepseek-chat

```

Listing 4: CLI 命令概览

4.2 validate 命令

验证 Manifest 文件的模式完整性 (ID 唯一性、DAG 依赖、路径有效性、参数合约和核函数引用)。

```

1 lamet-agent validate examples/pion_pdf_cg_manifest.json
2 # 输出: {"run_id": "...", "stages": [...], "correlator_count":
    ..., "status": "valid"}

```

Listing 5: validate 命令用法

4.3 plan 命令

交互式规划模式，接受不完整的 JSON/JSONC Manifest 并运行 LLM 控制的规划循环。

```
1 # 使用API后端
2 lamet-agent plan draft.jsonc --backend api --model deepseek/
   deepseek-chat
3
4 # 使用Codex后端
5 lamet-agent plan draft.jsonc --backend codex --model
   CODEX_MODEL_ID
```

Listing 6: plan 命令用法

规划循环执行以下操作：检查 Manifest、审查 HDF5 输入、提出终端问题、应用 JSON Patch 编辑，并在接受时写入 quick 和 full 两个版本的 Manifest 文件。

4.4 run 命令

运行阶段化的智能体循环，执行 Manifest 中定义的所有阶段。

```
1 lamet-agent run <manifest.json> \
2     --backend api \                               # LLM后端: mock|external|
   api|codex
3     --model deepseek/deepseek-chat \             # 模型ID (provider/
   model_id)
4     --verbose \                                   # 打印每个LLM周期
5     --max-tool-steps 40 \                         # 每个阶段最大LLM/工具周期
   数
6     --report_language en \                       # 报告语言: en|ch
7     --api-key-file api.key \                     # API密钥文件
8     --base-url https://api.example.com # 自定义API端点
```

Listing 7: run 命令完整参数

4.5 后端模式

表 2: LLM 后端模式说明

后端	说明	参数
mock	确定性测试支架，不使用真实 LLM	–
external	JSONL 转录回放（测试和回归）	--actions-path
api	OpenAI 兼容 HTTP API	--model provider/model_id, --api-key-file, --base-url
codex	Codex Python SDK	--model MODEL_ID

5 Manifest 系统

5.1 Manifest 概述

Manifest 是声明式 JSON/JSONC 文件，定义完整的分析流水线。它由三个顶级块组成：

1. **metadata**: 运行级设置(run_id、root_directory、artifacts_directory、target_observable、resample_mode、random_seed、workers、stages 列表)
2. **inputs**: 数据源定义 (correlators、artifacts、kernels)
3. **stages**: 阶段默认参数和作业列表

5.2 metadata 字段

表 3: metadata 字段说明

字段	必需	说明
run_id	是	运行标识符
root_directory	是	根目录，相对路径基于此解析
artifacts_directory	是	中间数据制品输出目录
target_observable	是	目标可观测测量: "pdf"、"da" 或"gpd"
resample_mode	是	重采样模式: "jk" (jackknife) 或"bs" (bootstrap)
random_seed	是	随机种子 (用于所有重采样步骤)
bs_samples	条件	bootstrap 样本数 (resample_mode="bs" 时必需)
sample_error_mode	否	误差模式: "covariance" (默认)、"mean" 或"median"
bin_size	否	重采样前的配置装箱大小
workers	否	样本拟合工作进程数 (默认 1)
stages	是	有序的阶段 ID 列表
parton	条件	部分子类型: "gluon" 或"quark"

5.3 inputs 字段

```

1  {
2    "inputs": {
3      "correlators": [
4        {
5          "correlator_id": "ensemble_2pt",
6          "correlator_type": "2pt",
7          "data_path": "data/ensemble_2pt.h5",
8          "ensemble": "HISQa060_X",
9          "hadron": "pion",
10         "gfix": "CG",
11         "source_operator": "g5",
12         "sink_operator": "g5",
13         "volume": "S48T64",
14         "lattice_spacing_fm": 0.0574,

```

```

15     "momentum": ["PX0PY0PZ0", "PX5PY0PZ0"]
16 },
17 {
18     "correlator_id": "ensemble_3pt",
19     "correlator_type": "3pt",
20     "data_path": "data/ensemble_free_3pt.h5",
21     "ensemble": "HISQa060_X",
22     "hadron": "pion",
23     "gfix": "CG",
24     "source_operator": "g5",
25     "sink_operator": "g5",
26     "current_operator": "gT_nonlocal",
27     "bz_direction": "X",
28     "volume": "S48T64",
29     "lattice_spacing_fm": 0.0574,
30     "momentum": ["PX0PY0PZ0", "PX5PY0PZ0"],
31     "tsep": [8, 10, 12],
32     "bT": [0],
33     "bz": [0, 1, 2]
34 }
35 ],
36 "artifacts": [
37     {
38         "id": "bare_input",
39         "stage": "correlator_analysis",
40         "path": "artifacts/correlator_analysis/ca_p5.nc",
41         "momentum": "PX0PY0PZ0",
42         "volume": "S48T64",
43         "lattice_spacing_fm": 0.0574
44     }
45 ],
46 "kernels": [
47     {
48         "stage": "renormalization",
49         "kernel_id": "ZMSbar_pdf",
50         "kernel_path": "src/lamet_agent/kernels.py",
51         "scheme": "hybrid_self_renormalization",
52         "kernel_parameters": { "mu": 2.0 }
53     }
54 ]

```

```

55     }
56 }

```

Listing 8: inputs 结构示例

5.4 标准 HDF5 关联函数格式

关联函数数据以 HDF5 格式存储，遵循以下路径约定：

- 普通 2pt: <source_op>/<sink_op>/<momentum>, 形状 (Lt, n_cfg)
- qDA 2pt: <source>/<sink>/<momentum>/bT<bT>/bz<bz>, 形状 (Lt, n_cfg)
- 3pt: <source>/<sink>/<current>/<momentum>/tsep<tsep>/bT<bT>/bz<bz>, 形状 (tsep+1, n_cfg)

动量标签格式为 PXnPYnPZn, 每个分量均为带符号整数。物理动量由下式导出：

$$p [\text{GeV}] = \frac{2\pi\hbar c}{L_s a [\text{fm}]} \sqrt{n_x^2 + n_y^2 + n_z^2}, \quad \hbar c = 0.1973269804 \text{ GeV fm.} \quad (1)$$

6 五阶段分析工作流

6.1 Stage 1: correlator_analysis (关联函数分析)

关联函数分析阶段从原始 HDF5 数据中提取裸矩阵元。

核心工具：

- inspect_correlator_scale: 选择关联函数缩放
- tune_ground_state: 2pt 拟合窗口扫描和模型平均
- tune_bare_matrix: 扫描裸矩阵元拟合窗口
- fit_bare_matrix_grid: 对所有 z 和所有重采样应用统一拟合窗口

拟合策略：

- 3pt_ratio: 通过 3pt/2pt 比值拟合提取裸矩阵元
- FH: Feynman-Hellmann 方法
- 3pt_ratio+FH: 组合方法
- qda_ratio: 准分布振幅比值拟合

拟合形式:

- **Breit** (默认): 初末态动量大小相等
- **NonBreit**: 允许不同的初末态动量

模型平均: 当 `model_average: true` 时, 使用 logGBF 权重组合多个拟合函数候选 (不同 `nstate` 和 `prior_width`)。

6.2 Stage 2: renormalization (重整化)

支持三种重整化方案:

6.2.1 Ratio 方案

简单的点对点比值重整化:

$$h_s^R(z) = \frac{h_s^{\text{target}}(z)}{h_s^{\text{denominator}}(z)}. \quad (2)$$

6.2.2 Hybrid-Ratio 方案

混合比值重整化, 在短距离使用比值方案, 在长距离施加指数修正。需要 `zs_fm` 参数定义切换距离。短距离采用固定分母 (如零动量), 长距离采用指数修正。

6.2.3 混合自重重整化方案

`scheme: "hybrid_self_renormalization"` 通过拟合零动量参考数据并结合短距离 $\overline{\text{MS}}$ 匹配来确定有限重整化:

$$g(z) - \ln Z_{\overline{\text{MS}}}^{\text{PDF}}(z; \mu) \simeq m_0 z + b, \quad z_R(z, a) = \exp[\ln M_{\text{fit}}(z, a) - g(z) + m_0 z]. \quad (3)$$

重整化矩阵元为:

$$H_{\text{ren}}(z) = \frac{H_{\text{bare}}(z)}{z_R(z, a) Z_{\overline{\text{MS}}}(z)}. \quad (4)$$

表 4: 混合自重整化参数

参数	位置	说明
<code>scheme_parameters.LambdaQCD_gev</code>	fit/apply job	Λ_{QCD} (GeV), 必需
<code>scheme_parameters.d</code>	fit job	连续/离散化系数 (参考算符值), 必需
<code>scheme_parameters.m0_gev</code>	apply job	目标算符 m_0 (可选, 用于跨算符)
<code>mu</code>	defaults/job	重整化标度, 默认 2.0 GeV
<code>scheme_parameters.svdcut</code>	fit job	SVD 截断, 默认 10^{-12}
<code>scheme_parameters.z_coverage_policy</code>	apply job	覆盖策略: extrapolate/strict/intersection

6.3 Stage 3: `fourier_transform` (傅里叶变换)

将重整化后的坐标空间矩阵元变换到动量空间, 得到准 PDF。支持多种傅里叶变换方法和外推方案。

核心工具:

- `load_renormalized_matrix_element_samples`: 加载重整化矩阵元
- `run_fourier_transform`: 执行傅里叶变换
- `plot_fourier_result`: 绑制结果图
- `plot_fourier_extension_quality_result`: 评估外推质量
- `report_fourier_result`: 生成阶段报告

6.4 Stage 4: `perturbative_matching` (微扰匹配)

通过 NLO 微扰匹配核将准 PDF 转换为光锥 PDF。

核心工具:

- `load_quasi_pdf`: 加载准 PDF 数据
- `build_matching_kernel`: 构建匹配核 (支持 ratio 和 hybrid 方案)
- `apply_matching`: 应用匹配核
- `plot_matched_pdf`: 绑制匹配后 PDF
- `report_matching_result`: 生成匹配报告

6.5 Stage 5: extrapolation (外推)

执行连续极限和无穷大动量外推。

6.6 阶段结束后处理

每个阶段完成后, `reporting.py` 模块以选定语言 (en 或 ch) 生成报告, 帮助用户追踪分析进展和审查中间结果。

7 NetCDF 中间数据格式

7.1 EnsembleData 格式

阶段间通过 EnsembleData NetCDF 文件传递数据, 路径为 `<artifacts_directory>/<stage>/<data>`。
数据布局:

- **首维:** `resample` (bootstrap/jackknife 样本索引, 或 `gvar` 的长度 1)
- **物理维度:** 如 z (坐标空间) 或 x (动量空间)
- **属性:** 保留的 `ensemble` 和 `resample` 元数据, 加上阶段特定的 `attrs`

7.2 典型制品链

表 5: 阶段制品示例

阶段	制品示例
<code>correlator_analysis</code>	<code>correlator_analysis/ca_p5.nc</code>
<code>renormalization</code>	<code>renormalization/rn_p5.nc</code>
<code>fourier_transform</code>	<code>fourier_results/fourier_result.nc</code> , <code>fourier_results/fourier_fit.nc</code>
<code>perturbative_matching</code>	<code>matching_results/quasi_pdf.nc</code>

7.3 Python 读写

```

1 from lamet_agent.core.data import EnsembleData
2
3 # 写入
4 data.to_netcdf("artifacts/fourier_results/fourier_result.nc")
5
6 # 读取
```

```
7 reload = EnsembleData.from_netcdf("artifacts/fourier_results/
   fourier_result.nc")
```

Listing 9: NetCDF 数据读写

复数数组自动往返 (auto_complex=True), 无需手动拆分实部和虚部。

7.4 外部检查

NetCDF 是自描述格式, 可使用标准工具检查:

```
1 # 命令行检查
2 ncdump -h file.nc
3
4 # Python / xarray 读取
5 import json, xarray as xr
6 from lamet_agent.core.data import EnsembleInfo
7
8 da = xr.load_dataarray("fourier_result.nc", auto_complex=True)
9 ensemble = EnsembleInfo(**json.loads(da.attrs["ensemble"]))
10 values = da.values # shape (n_sample, *physical_dims)
```

Listing 10: 使用外部工具检查 NetCDF

8 示例演示

8.1 示例 1: 介子 PDF CG 规范完整 workflow

```
1 # 验证 Manifest
2 lamet-agent validate examples/pion_pdf_cg_manifest.json
3
4 # 运行完整流水线 (使用 API 后端)
5 lamet-agent run examples/pion_pdf_cg_manifest.json \
6     --backend api \
7     --model deepseek/deepseek-chat \
8     --verbose
9
10 # 运行 (使用 Codex 后端)
11 lamet-agent run examples/pion_pdf_cg_manifest.json \
12     --backend codex \
13     --model CODEX_MODEL_ID \
```



```
14 --verbose
```

Listing 11: 运行 介子 PDF 完整工作流

此工作流通过关联函数分析、hybrid-ratio 重整化、傅里叶变换和微扰匹配四个阶段，从 P0/P5 动量关联函数数据出发，最终得到 介子的胶子 PDF。

8.2 示例 2: 介子 DA GI 规范完整工作流

```
1 lamet-agent validate examples/pion_da_gi_manifest.json
2 lamet-agent run examples/pion_da_gi_manifest.json \
3     --backend api --model deepseek/deepseek-chat --verbose
```

Listing 12: 运行 介子 DA 工作流

此工作流从 qDA 关联函数分析开始，经过重整化和匹配，最终得到 介子的分布振幅 (DA)。

8.3 示例 3: 混合自重重整化

```
1 {
2   "stages": {
3     "renormalization": {
4       "defaults": {
5         "normalization": false,
6         "scheme": "hybrid_self_renormalization",
7         "mu": 2.0,
8         "scheme_parameters": { "LambdaQCD_gev": 0.1 }
9       },
10      "jobs": [
11        {
12          "id": "rn_zR_fit",
13          "inputs": { "reference": "bare_pdf_reference" },
14          "params": {
15            "scheme_parameters": { "d": -0.08183 }
16          }
17        },
18        {
19          "id": "rn_da_a06",
20          "inputs": { "target": "bare_da_a06", "zR": "rn_zR_fit" },
21          "params": {
```

```
22         "scheme_parameters": { "d": 0.19, "m0_gev": -0.094 }
23     }
24 }
25 ]
26 }
27 }
28 }
```

Listing 13: 混合自重整化 Manifest 片段

作业角色:

- **reference** (fit job): 指向裸零动量 EnsembleData
- **target** (apply job): 指向待重整化的裸矩阵元
- **zR** (apply job): 指向 fit job 的 NetCDF 输出

8.4 示例 4: 交互式规划

```
1 # 从不完整的草稿Manifest开始
2 lamet-agent plan draft_manifest.jsonc --backend api --model
   deepseek/deepseek-chat
3
4 # 规划循环会:
5 # 1. 检查Manifest完整性
6 # 2. 列出缺失参数、不一致设置、建议修改和数据转换
7 # 3. 写入quick (快速烟雾测试) 和full (完整生产) 两个版本
```

Listing 14: 交互式规划流程

9 代码架构详解

9.1 agent.py —run_agent()

`run_agent()` 是主入口函数, 执行 `metadata.stages` 中定义的阶段列表。对每个阶段作业:

1. `validate_stage_inputs()` 检查缺失的输入
2. `build_stage_static_prompt()` 一次性组装静态上下文
3. `make_llm_session()` 创建可插拔的 LLM 会话

4. 多轮循环 (最多 `max_tool_steps`, 默认 40 轮)
5. 每轮 LLM 发出一个 JSON 动作, `prepare_tool_args()` 和 `resolve_stage_tools()` 运行工具
6. 终止工具将主数据放入 `store["output"]`

9.2 *manifest.py* —模式定义

定义 `AnalysisManifest` 的三部分结构: `metadata`、`source inputs` 和 `stage-job` 模式。验证 ID 唯一性、有序作业引用和根相对路径。

9.3 *core/llm.py* —LLM 会话后端

提供四种可插拔的 `LlmSession` 后端:

- **MockSession**: 确定性测试支架
- **ExternalSession**: JSONL 转录回放
- **CodexSession**: Codex Python SDK
- **ApiSession**: OpenAI 兼容 HTTP (支持 DeepSeek、OpenAI 等提供商)

`PROVIDERS` 字典定义每个提供商的 base URL、默认模型和 API 密钥环境变量。

9.4 *core/tools.py* —工具系统

`STAGE_TOOLS` 注册表将阶段 ID 映射到具体的工具函数。`prepare_tool_args()` 注入 `Manifest` 路径、强制绑图输出到 `artifacts/`、填充运行时元数据。

关键函数:

- `resolve_stage_tools(stage)`: 返回阶段的工具注册表
- `resolve_job_tools(stage, job, params)`: 返回当前作业合约有效的工具子集
- `prepare_tool_args(tool_name, args, ...)`: 准备工具参数
- `filter_tool_kwargs(tool, args)`: 过滤 LLM 提供的无效参数
- `validate_stage_inputs(stage, manifest, job)`: 验证阶段输入

9.5 core/data.py —数据容器

```

1  class EnsembleInfo:
2      """系综元数据"""
3      id: str          # 系综标识符
4      volume: str      # 格点体积 (如"S48T64")
5      a_fm: float      # 格点间距 (fm)
6      # ... 更多字段
7
8  class EnsembleData:
9      """重采样后的格点数据"""
10     values: np.ndarray      # shape (n_sample, *
        physical_dims)
11     physical_dims: tuple[str] # 物理维度名
12     coords: dict[str, np.ndarray] # 坐标值
13     ensemble: EnsembleInfo    # 系综信息
14     resample_mode: str        # "jk" 或 "bs"
15     attrs: dict[str, Any]     # 额外属性
16
17     def to_netcdf(self, path: str) -> None: ...
18     @classmethod
19     def from_netcdf(cls, path: str) -> EnsembleData: ...

```

Listing 15: 数据容器接口

10 API 接口清单

10.1 CLI 接口

```

1  # validate
2  lamet-agent validate <path: Path>
3
4  # plan
5  lamet-agent plan <manifest: Path> \
6      --backend <api|codex|mock> \
7      --model <provider/model_id> \
8      [--api-key-file <path>] \
9      [--base-url <url>]
10
11 # run

```

```

12 lamet-agent run <manifest: Path> \
13     --backend <mock|external|api|codex> \
14     [--model <provider/model_id>] \
15     [--actions-path <path>] \           # external 后端
16     [--api-key-file <path>] \         # api 后端
17     [--base-url <url>] \             # api 后端
18     [--verbose | -v] \
19     [--max-tool-steps <int>] \         # 默认 40
20     [--report_language <en|ch>]      # 默认 en

```

Listing 16: CLI 命令签名

10.2 Agent 核心 API

```

1 def run_agent(
2     manifest: AnalysisManifest,
3     *,
4     backend: str,
5     actions_path: Path | None = None,
6     provider: str | None = None,
7     model_name: str | None = None,
8     api_key: str | None = None,
9     base_url: str | None = None,
10    verbose: bool = False,
11    max_tool_steps: int = 40,
12    report_language: str = "en",
13 ) -> dict[str, Any]:
14     """执行完整的阶段化智能体循环。
15     返回: {run_id, status, stages, completed_stages,
16           stage_reports, ...}
17     """

```

Listing 17: agent.py 核心函数

10.3 Manifest API

```

1 class AnalysisManifest:
2     """完整的分析Manifest"""
3     run_id: str
4     metadata: RunMetadata

```

```

5     inputs: AnalysisInputs
6     stages: dict[str, StageSpec]
7
8     @property
9     def root_directory(self) -> Path: ...
10    @property
11    def correlators(self) -> list[CorrelatorInput]: ...
12    @property
13    def kernels(self) -> list[KernelInput]: ...
14
15    def validate_manifest_file(path: Path) -> AnalysisManifest: ...
16    # 验证并加载Manifest文件，失败时抛出异常
17
18    class CorrelatorInput:
19        """关联函数输入定义"""
20        correlator_id: str
21        correlator_type: Literal["2pt", "3pt"]
22        data_path: str
23        ensemble: str
24        hadron: str
25        gfix: str
26        source_operator: str
27        sink_operator: str
28        volume: str
29        lattice_spacing_fm: float
30        momentum: list[str]
31        tsep: list[int] | None          # 3pt only
32        bT: list[int] | None           # 3pt only
33        bz: list[int] | None           # 3pt only
34        bz_direction: str | None       # 3pt only
35        current_operator: str | None   # 3pt only
36
37    class StageJob:
38        """阶段作业定义"""
39        id: str
40        inputs: dict[str, str]          # 角色 -> 上游作业ID
41        params: dict[str, Any]          # 作业级参数 (合并defaults后)

```

Listing 18: manifest.py 核心接口

10.4 数据 API

```

1  class EnsembleData:
2      """重采样格点数据容器"""
3      values: np.ndarray
4      ensemble: EnsembleInfo
5      resample_mode: str
6      attrs: dict[str, Any]
7
8      def to_netcdf(self, path: str | Path) -> None: ...
9      @classmethod
10     def from_netcdf(cls, path: str | Path) -> EnsembleData: ...
11
12  class EnsembleInfo:
13      """系综元数据"""
14      id: str
15      volume: str
16      a_fm: float
17      # 支持序列化和反序列化
18      def to_dict(self) -> dict[str, Any]: ...
19      @classmethod
20      def from_dict(cls, data: dict[str, Any]) -> EnsembleInfo: ...

```

Listing 19: core/data.py 核心接口

10.5 LLM API

```

1  def parse_api_model(model_spec: str) -> tuple[str, str]:
2      """解析 'provider/model_id' 格式的模型规格。
3      返回: (provider, model_id)
4      """
5
6  def provider_config(provider: str) -> dict[str, str] | None:
7      """获取提供商配置。
8      返回: {base_url, default_model, key_env}
9      """
10
11  class LlmSession(ABC):
12      """LLM会话抽象基类"""
13      @abstractmethod

```

```

14     def send(self, messages: list[dict], tools: list[dict]) ->
        dict: ...
15     @abstractmethod
16     def close(self) -> None: ...
17
18 class ApiSession(LlmSession):
19     """OpenAI 兼容 API 会话"""
20     def __init__(self, provider: str, model: str, api_key: str,
        base_url: str | None): ...
21
22 class MockSession(LlmSession):
23     """Mock 会话 (测试用)"""
24     def __init__(self, actions: list[dict]): ...
25
26 class ExternalSession(LlmSession):
27     """外部 JSONL 转录回放"""
28     def __init__(self, actions_path: Path): ...
29
30 class CodexSession(LlmSession):
31     """Codex SDK 会话"""
32     def __init__(self, model: str | None = None): ...

```

Listing 20: core/llm.py 核心接口

10.6 工具系统 API

```

1 def resolve_stage_tools(stage: str) -> dict[str, Callable]:
2     """返回阶段的 STAGE_TOOLS 注册表"""
3
4 def resolve_job_tools(
5     stage: str, job: StageJob, effective_params: dict,
6     *, stage_tools: dict | None = None
7 ) -> dict[str, Callable]:
8     """返回当前作业合约有效的工具子集"""
9
10 def prepare_tool_args(
11     tool_name: str, args: dict, *,
12     manifest: AnalysisManifest, stage: str, job: StageJob,
13     effective_params: dict, artifacts_dir: Path, store: dict |
        None = None

```



```

14 ) -> dict[str, Any]:
15     """准备工具参数：解析路径、注入Manifest元数据、强制绑图输出位置"""
16
17 def validate_stage_inputs(
18     stage: str, manifest: AnalysisManifest, job: StageJob
19 ) -> list[str]:
20     """通过阶段的validate_stage_inputs辅助函数返回输入问题列表"""
21
22 def resolve_plot_save_path(
23     raw: str | None, *,
24     artifacts_dir: Path, default_stem: str,
25     root_directory: Path | None = None
26 ) -> str:
27     """解析绑图输出路径"""
28
29 def filter_tool_kwargs(
30     tool: Callable, args: dict
31 ) -> tuple[dict, dict]:
32     """过滤LLM提供的无效参数，只保留工具签名中的参数"""

```

Listing 21: core/tools.py 核心接口

10.7 阶段参数合约

```

1 STAGE_PARAM_CONTRACTS: dict[str, dict[str, Any]]
2 # 每个阶段的参数合约注册表。
3 # 在DAG执行前递归拒绝未知的defaults/params键。
4
5 # 支持的阶段ID:
6 #   "correlator_analysis" — 关联函数
7 #   "renormalization" — 重整化
8 #   "fourier_transform" — 傅里叶变换
9 #   "perturbative_matching" — 微扰匹配
10 #   "extrapolation" — 外推

```

Listing 22: manifest_params.py 核心接口

10.8 各阶段工具注册表

```

1  # correlator_analysis 阶段工具
2  - inspect_correlator_scale: 选择关联函数缩放
3  - tune_ground_state: 2pt拟合窗口扫描 + 模型平均
4  - tune_bare_matrix: 扫描裸矩阵元拟合窗口
5  - fit_bare_matrix_grid: 对所有z和样本应用统一拟合窗口
6
7  # renormalization 阶段工具
8  - apply_ratio_scheme_renormalization: 应用ratio/hybrid-ratio重整
    化
9  - fit_self_renormalization_factor: 拟合自重整化因子
10 - apply_self_renormalization: 应用自重整化
11 - load_bare_matrix_element: 加载裸矩阵元
12 - load_bare_matrix_element_grid: 加载裸矩阵元网格
13 - plot_renormalized_matrix_element: 绑制重整化矩阵元
14 - plot_self_renormalization_diagnostics: 绑制自重整化诊断图
15
16 # fourier_transform 阶段工具
17 - load_renormalized_matrix_element_samples: 加载重整化矩阵元
18 - run_fourier_transform: 执行傅里叶变换
19 - plot_fourier_result: 绑制傅里叶结果
20 - plot_fourier_extension_quality_result: 绑制外推质量图
21 - report_fourier_result: 生成傅里叶报告
22
23 # perturbative_matching 阶段工具
24 - load_quasi_pdf: 加载准PDF
25 - build_matching_kernel: 构建微扰匹配核
26 - apply_matching: 应用匹配
27 - plot_matched_pdf: 绑制匹配后PDF
28 - report_matching_result: 生成匹配报告

```

Listing 23: 各阶段工具

11 开发与贡献

11.1 运行测试

```

1  # 运行所有单元测试
2  pytest tests/unit/
3

```

```
4 # 运行特定测试文件
5 pytest tests/unit/test_agent.py -v
6 pytest tests/unit/test_stage_core.py -v
7 pytest tests/unit/test_schemas.py -v
```

Listing 24: 运行测试套件

11.2 添加新阶段

1. 在 `src/llm_agent/stages/` 下创建新目录
2. 实现 `prompts.py` (阶段指令文本)、`skills.py` (阶段检查和工具目录) 和 `functions.py` (工具函数和 `STAGE_TOOLS` 注册表)
3. 在 `core/stages.py` 中注册阶段 ID 到包名映射
4. 在 `manifest_params.py` 的 `STAGE_PARAM_CONTRACTS` 中定义参数合约
5. 如需要, 实现 `reporting.py` 生成阶段报告

11.3 添加新 LLM 提供商

在 `core/llm.py` 的 `PROVIDERS` 字典中添加新条目:

```
1 PROVIDERS = {
2     "deepseek": {
3         "base_url": "https://api.deepseek.com/v1",
4         "default_model": "deepseek-chat",
5         "key_env": "DEEPSEEK_API_KEY",
6     },
7     "openai": {
8         "base_url": "https://api.openai.com/v1",
9         "default_model": "gpt-4o",
10        "key_env": "OPENAI_API_KEY",
11    },
12    # 添加新的提供商...
13 }
```